

OpenRouter integration manual for mindX

OpenRouter is a single OpenAI-compatible HTTPS endpoint at `https://openrouter.ai/api/v1` that fronts ~400 models from ~60 upstream providers, normalizes their request/response schemas, transparently fails over on errors, and exposes a small but rich set of routing, caching, and reasoning extensions that no upstream provider offers natively. **For mindX, OpenRouter is the correct universal LLM backplane:** it speaks the OpenAI dialect that every Python framework already supports, eliminates per-provider auth ceremony, and gives the cognitive layer a `:free` tier suitable for non-critical agent work plus a uniform paid path for production. The catch — and it is a load-bearing catch — is that OpenRouter's free tier rotates aggressively (≥ 30 free models churned out between mid-2025 and May 2026), upstream providers like Chutes and Targon impose their own undocumented per-model limits on top of the global 20 RPM / 50–1000 RPD caps, and the response from a `:free` slug is **not** feature-equivalent to the paid slug of the same model (shorter context, no tool calling, weaker provider routing). This document specifies the production-grade architecture mindX must adopt to insulate itself from that volatility while exploiting OpenRouter's strengths. The narrative flows from raw protocol mechanics → free-model registry → rate-limit and reliability strategy → mindX adapter architecture → operational addenda.

Quickstart, authentication, and the three classes of API key

The base URL is `https://openrouter.ai/api/v1`. Chat completions live at `POST /chat/completions`, legacy completions at `POST /completions`, the model catalog at `GET /models`, generation lookup at `GET /generation?id=<gen-id>`, the credits/key introspection at `GET /key` and `GET /credits`, and key management at `/keys` (list/create/get/patch/delete). An OpenAI Responses-style endpoint also exists in beta at `POST /api/alpha/responses`. The full OpenAPI spec is published at `https://openrouter.ai/openapi.yaml` and `https://openrouter.ai/openapi.json` and should be considered the canonical machine-readable reference; mindX's adapter should fetch it at build time to regenerate Pydantic models rather than hand-code schemas. (OpenRouter + 2)

Authentication is an `Authorization: Bearer sk-or-v1-...` header on every request.

OpenRouter recognizes **three categories of key** that mindX must treat distinctly. **Standard inference keys**, created at `openrouter.ai/keys`, are what individual mindX agents present. **Management (a.k.a. provisioning) keys**, created at `openrouter.ai/settings/management-keys`, are the *only* keys that may call `POST /api/v1/keys` to mint, rotate, or disable inference keys; they cannot themselves perform completions. mindX's bankon.pythai.net identity layer should hold a single management key in a sealed Podman secret and provision a dedicated inference key per agent or per AgenticPlace tenant on demand, with a credit ceiling (`limit`) and `limit_reset: "monthly"`. **BYOK** (bring-your-own-key) lets a tenant attach their own Anthropic, OpenAI, Vertex, or Bedrock key inside `openrouter.ai/settings/integrations`;

OpenRouter then routes BYOK endpoints first regardless of `(provider.order)`, applies a 5 % surcharge against OpenRouter credits (waived for the first **1 million BYOK requests per month** since October 1 2025), and exposes `(is_byok: true)` on the response. mindX should expose BYOK as a per-tenant setting in AgenticPlace, falling back to the platform's shared inference pool when a tenant has not registered upstream credentials. `(openrouter + 7)`

Optional but recommended request headers are `(HTTP-Referer)` (your site URL — drives the openrouter.ai/rankings leaderboard), `(X-Title)` (alias `(X-OpenRouter-Title)`, your app name), `(X-OpenRouter-Categories)` for marketplace classification, and `(x-session-id)` (≤ 256 chars) to group related requests for observability — overridden by the body's `(session_id)` field when both are present. The Anthropic-specific passthrough `(x-anthropic-beta)` (comma-separated) unlocks `(fine-grained-tool-streaming-2025-05-14)`, `(interleaved-thinking-2025-05-14)`, and `(structured-outputs-2025-11-13)` — without that last header OpenRouter **silently strips** `(strict: true)` from your tool definitions, a behaviour change that landed November 13 2025 and broke many existing strict-mode integrations. On the response, OpenRouter always returns the `(X-Generation-Id)` header which is identical to the `(id)` field in the JSON body and is the audit key for `(GET /api/v1/generation?id=...)`. `(OpenRouter + 5)`

The chat completions request and response schemas, in full

A `(POST /chat/completions)` body extends OpenAI's schema with seven OpenRouter-only fields: `(models)` (an ordered fallback array tried on any error), `(route: "fallback")` (legacy alias for the same), `(provider)` (the routing-preferences object detailed below), `(transforms)` (currently only `(["middle-out"])`), `(plugins)` (web, file-parser, response-healing, context-compression, auto-router), `(reasoning)` (effort/budget/exclude/enabled), and `(usage: {include: true})` (now a no-op — usage is always returned). Standard OpenAI fields all work: `(model)`, `(messages)` or `(prompt)`, `(stream)`, `(max_tokens)` (deprecated in favour of `(max_completion_tokens)`, but still accepted; some providers enforce a minimum of 16), `(temperature)` (0-2), `(top_p)` (0-1), `(top_k)` (≥ 1 , not on OpenAI), `(frequency_penalty)` and `(presence_penalty)` (-2-2), `(repetition_penalty)` (0-2), `(min_p)` (0-1), `(top_a)` (0-1), `(seed)`, `(stop)`, `(logit_bias)`, `(logprobs)`, `(top_logprobs)` (0-20, requires `(logprobs: true)`), `(response_format)`, `(tools)`, `(tool_choice)`, `(parallel_tool_calls)` (default `(true)`), `(prediction)`, `(user)`, `(metadata)` (≤ 16 string pairs, key ≤ 64 chars, value ≤ 512 chars, no brackets in keys), `(service_tier)` (`(auto)`/`(default)`/`(flex)`/`(priority)`/`(scale)`), `(modalities)` (`(text)`/`(image)`/`(audio)`), `(image_config)`, `(verbosity)` (`(low)`/`(medium)`/`(high)`/`(xhigh)`/`(max)`), `(cache_control)`, `(stream_options)`, `(structured_outputs)` (boolean hint), and `(zdr)` (boolean — OR'd with the account-level toggle, cannot be disabled per request). `(PydanticAI + 19)`

Messages take the standard `({role, content})` shape but `(content)` can be either a string or an array of content parts. Text parts are `({type: "text", text})`. Image parts are `({type: "image_url", image_url: {url, detail?}})` where `(url)` may be HTTPS or a base64 data URL (`(data:image/png;base64,...)`). PDF/file parts are `({type: "file", file: {filename,`

`file_data}}` and are accepted on **every** model — OpenRouter selects a parsing engine via the `file-parser` plugin (`native` for multimodal models, `mistral-ocr` at \$2/1000 pages for scans, `cloudflare-ai` free for text-only PDFs). Tool calls use the OpenAI shape: `tools: [{type: "function", function: {name, description, parameters}}]`, with `tool_choice` accepting `"auto"`, `"none"`, `"required"`, or `{type: "function", function: {name}}`. (OpenRouter + 3)

The non-streaming response is a standard OpenAI envelope plus several OpenRouter additions. Each `choices[]` entry has both `finish_reason` (normalized to `stop`/`length`/`tool_calls`/`content_filter`/`error`) and `native_finish_reason` (the raw upstream value). The `usage` object contains the usual `prompt_tokens`/`completion_tokens`/`total_tokens` plus `prompt_tokens_details: {cached_tokens, cache_write_tokens, audio_tokens?, video_tokens?}`, `completion_tokens_details: {reasoning_tokens?, audio_tokens?, image_tokens?}`, `cost` (in credits = USD), `is_byok`, `cost_details` (especially `upstream_inference_cost` for BYOK), and `server_tool_use: {web_search_requests?}`. The top-level `model` field reports the **actually-used** model (relevant when `models[]` fallback or `openrouter/auto` redirects), and an undocumented but consistently-present top-level `provider` string names the upstream that served the request. **mindX must log both `model` and `provider` on every completion** — without them, regression triage becomes guesswork because Auto-Exacto and load-balancing can silently swap providers between identical requests. (OpenRouter + 5)

Streaming uses Server-Sent Events at the same endpoint with `stream: true`. Frames are `data: {json}\n\n` ending with `data: [DONE]`. **Two SSE-parsing gotchas** must be handled: lines that start with `:` are SSE *comment* keep-alives (concretely `: OPENROUTER PROCESSING\n\n`) and must be skipped before any JSON parsing — naïve `JSON.parse` consumers crash on them; and **mid-stream errors keep HTTP at 200** because headers were already flushed, so an upstream failure surfaces as an SSE event with a top-level `error` field and `finish_reason: "error"` on the final delta. The final usage chunk is always emitted **once before** `[DONE]` with an empty `choices` array; the legacy `stream_options: {include_usage: true}` and body-level `usage: {include: true}` flags are **deprecated no-ops** as of late 2025 — usage now ships automatically. Aborting the HTTP connection cancels billing on most providers (OpenAI, Anthropic, Fireworks, DeepInfra, Together, xAI, DeepSeek, Chutes, etc.) but **not** on AWS Bedrock, Groq, Google AI Studio, Mistral, Perplexity, Replicate, HuggingFace, Targon, or several others — for those, partial output is still billed. (OpenRouter + 9)

The error envelope is uniform: `{error: {code, message, metadata??}}` where `code` typically equals the HTTP status. The full enumeration is **400** (bad request, CORS), **401** (invalid/disabled key, expired OAuth session), **402** (insufficient credits — fires even on `:free` models when balance is negative), **403** (moderation rejection — `metadata` carries `reasons[]`, `flagged_input` truncated to ≤100 chars, `provider_name`, `model_slug`), **404** (deprecated model, "no endpoints for this model found"), **408** (timeout), **413** (content too large), **422** (unprocessible entity), **429** (rate limited), **500/502/503** (internal/upstream-down/no-provider-

meets-routing-constraints). Provider-side errors also carry `metadata.provider_name` and `metadata.raw`. On the Responses API, several would-be errors are normalized to successes with `finish_reason: "length": context_length_exceeded, max_tokens_exceeded, token_limit_exceeded, string_too_long`. The **"no content generated"** mode — zero completion tokens, null finish reason — historically billed prompt-processing; since December 2024 OpenRouter's **Zero Completion Insurance** waives both prompt and completion charges on these and on `finish_reason: "error"` responses across all models, automatically, free.

[GitHub + 9](#)

The free-model catalog as it stands in May 2026

The free tier has been savaged. A LiteLLM cleanup issue (BerriAI/litellm #20521, February 5 2026) confirmed 39 OpenRouter slugs were removed from `/api/v1/models`, including most of the Llama 3.1/3.2/4 family, every Mistral `:free`, every DeepSeek `:free` (R1, R1-0528, R1-Distill-*, V3, V3.1, Chat, Chimera, Prover-V2), every Microsoft Phi `:free`, `microsoft/mai-ds-r1:free`, the Moonshot Kimi `:free` family, `x-ai/grok-4-fast:free`, several NVIDIA Llama-Nemotron `:free` variants, and the cloaked alphas (Optimus, Quasar, Horizon, Sonoma). The collection page at `openrouter.ai/models?max_price=0` plus the CostGoat aggregator dump of May 3 2026 enumerate **33 currently-free entries**. mindX must therefore **never hard-code the free list**; the model registry should poll `GET /api/v1/models` every six hours, filter for `pricing.prompt == "0" AND pricing.completion == "0"` or `id.endswith(":free")`, and reconcile against a persistent `free_model_history` table in mindX's storage so deprecations are detected and routed away from gracefully. [github](#)

The currently-stable free roster groups as follows. **Meta** retains only `meta-llama/llama-3.3-70b-instruct:free` (65 K context — half the paid 131 K — text-only with tool calling, multilingual; the de facto safe default for general chat) and `meta-llama/llama-3.2-3b-instruct:free` (131 K, text-only). **Google** holds the most slots: `google/gemma-4-31b-it:free` (256 K, vision + tools + structured outputs + thinking, Apache 2.0 — the strongest free vision/agent model), `google/gemma-4-26b-a4b-it:free` (256 K, MoE 25.2 B / 3.8 B active, vision + video, tools, thinking, Apache 2.0), `google/gemma-3-27b-it:free` (131 K, vision, no tools), `google/gemma-3-12b-it:free` and `google/gemma-3-4b-it:free` (32 K, vision), and the mobile-tuned `google/gemma-3n-e4b-it:free` and `google/gemma-3n-e2b-it:free` (8 K, text-only). The two `google/lyria-3-*-preview` entries are music-generation models that emit audio, not chat tokens, and must be excluded from the chat registry. **Qwen/Alibaba** kept only `qwen/qwen3-coder:free` (262 K, 480 B / 35 B active MoE — the strongest free coding model) and `qwen/qwen3-next-80b-a3b-instruct:free` (262 K, 80 B / 3 B active). **NVIDIA** is the heaviest free presence with `nvidia/nemotron-3-super-120b-a12b:free` (262 K, 120 B / 12 B active hybrid Mamba-Transformer), `nvidia/nemotron-3-nano-omni-30b-a3b-reasoning:free` (256 K, true text+image+video+audio omni-modal with a 16 384-token reasoning budget), `nvidia/nemotron-3-nano-30b-a3b:free` (256 K text-only), `nvidia/nemotron-nano-12b-v2-`

`vl:free` (128 K vision-language), and `nvidia/nemotron-nano-9b-v2:free` (128 K, dual reasoning/non-reasoning toggled via system prompt). **OpenAI** open-weights surfaces as `openai/gpt-oss-120b:free` (131 K, 117 B / 5.1 B active, Apache 2.0, configurable reasoning effort, function calling, structured outputs, browsing) and `openai/gpt-oss-20b:free` (131 K, 21 B / 3.6 B active, edge-deployable in 16 GB). **Z.AI** holds `z-ai/glm-4.5-air:free` (131 K, hybrid thinking via `reasoning.enabled`). **Specialty/experimental** entries include `inclusionai/ling-2.6-1t:free` (262 K, 1 T params), `tencent/hy3-preview:free` (256 K, 295 B / 21 B MoE — time-limited 2-week launch window so likely already paid), `minimax/minimax-m2.5:free` (197 K, productivity-tuned, 80.2 % SWE-Bench Verified), `poolside/laguna-m.1:free` and `poolside/laguna-xs.2:free` (131 K, 8 K max output, coding agents), `nousresearch/hermes-3-llama-3.1-405b:free` (131 K, the largest free model by base parameter count), `cognitivecomputations/dolphin-mistral-24b-venice-edition:free` (32 K, uncensored fine-tune), the edge models `liquid/lfm-2.5-1.2b-{thinking,instruct}:free` (32 K), and `baidu/qianfan-ocr-fast:free` (66 K, OCR-only — exclude from chat registry). The cloaked `openrouter/owl-alpha` (1.05 M context, text-only, tool calling) explicitly logs prompts and completions for upstream training; mindX must treat it as a "non-confidential dev only" tier and never route customer data to it. The meta-router `openrouter/free` randomly selects an available free model that meets capability requirements — useful as a final fallback but not as a primary route because it offers no SLA and 200 K declared context can collapse to whichever underlying model wins the dispatch. `costgoat + 17`

OpenRouter's own FAQ states bluntly that **free models "are usually not suitable for production use"** and "may be removed or have limits adjusted without notice." mindX's policy must mirror that: `:free` for development, batch evaluation, agent self-criticism passes, embedding-style classification, and any task where a 30-second hiccup is acceptable; paid for user-facing, latency-sensitive, or revenue-bearing paths. `OpenRouter` `Costgoat`

Rate limits, the `$10 unlock`, and what 429 actually means

The official policy is sparse and clear. Free models are governed at **20 RPM** across all `:free` slugs combined, with a daily cap of **50 requests for accounts that have purchased less than \$10 in lifetime credits** and **1 000 requests per day for accounts that have ever purchased $\geq$ \$10** — and that higher cap **persists even after the balance drops below \$10**. The `$10 unlock` is the single highest-leverage knob in OpenRouter operations and mindX should perform a one-time \$10 credit deposit on every production OpenRouter account. Paid models have **no platform-level RPM/RPD** (an Oreate AI Blog claim of `$1 = 1 RPS, max 500 RPS` is **unconfirmed** and contradicted by the official pricing page); only upstream provider limits and Cloudflare's DDoS layer apply. Limits are **globally governed per account** — minting more keys or creating sibling accounts does **not** raise capacity. Failed requests count toward the daily quota, so a 429 storm rapidly self-amplifies. Quotas reset at 00:00 UTC. `OpenRouter + 9`

The real rate-limit landscape is more textured. Upstream providers impose their own per-model limits **on top of** OpenRouter's account caps. The 429 body for `google/gemini-2.0-flash-thinking-exp:free` shows `X-RateLimit-Limit: 80` — Google AI Studio's 80 RPD per-model cap, far below OpenRouter's 1 000 RPD. The Chutes provider, which during 2025 became the *only* free upstream for the entire DeepSeek `:free` family, formally rate-limits OpenRouter free traffic to protect its paying subscribers (announcement July 22 2025). Targon similarly throttles aggressively; Janitor AI's official troubleshooting guide tells users to add Targon to OpenRouter Settings → Ignored Providers when 429s persist on `meta-llama/llama-3.3-70b-instruct:free`. Peak hours ($\approx$ 14:00–22:00 UTC, US business time) saturate Chutes-hosted DeepSeek free; the calm window is 02:00–06:00 UTC. [GitHub + 4](#)

A 429 from OpenRouter has a specific shape that mindX must parse. The HTTP status is `429 Too Many Requests` (except mid-stream where it stays 200 and arrives as an SSE event). The body looks like `{"error": {"message": "Rate limit exceeded: limit_rpd/<model-slug>/<id>", "code": 429, "metadata": {"headers": {"X-RateLimit-Limit": "80", "X-RateLimit-Remaining": "0", "X-RateLimit-Reset": "1741305600000"}}}}`. Three operational facts follow. First, `X-RateLimit-Reset` is **epoch milliseconds, not seconds and not delta-seconds** — confirmed at `1741305600000` = 2025-03-07 00:00:00 UTC. Second, the bucket type is encoded in the message string: `limit_rpd/...` for the daily bucket, `limit_rpm/...` for per-minute. Third, these `X-RateLimit-*` fields live **inside the JSON body's `error.metadata.headers`**, not as actual HTTP headers — a client reading only `response.headers` will miss them. A separate provider-upstream 429 shape (`{error: {code: 429, message: "Provider returned error", metadata: {raw: "moonshotai/kimi-k2:free is temporarily rate-limited upstream...", provider_name: "Chutes"}}`) signals an upstream cap rather than your account hitting OpenRouter limits, and is the correct trigger for swapping providers via `provider.ignore`. [GitHub](#) [GitHub](#)

For self-discovery, `GET /api/v1/key` returns a `data` object with `label`, `limit`, `limit_reset`, `limit_remaining`, `include_byok_in_limit`, `usage`/`usage_daily`/`usage_weekly`/`usage_monthly`, the BYOK equivalents, and `is_free_tier` — the canonical signal for whether your account is in the 50 RPD or 1 000 RPD bucket. Note that the legacy `rate_limit` sub-object is deprecated and no longer accurate; **there is no API to introspect remaining free-tier request counts**, only credit usage. mindX must therefore maintain its own client-side counters keyed by `(account, model_class)` and publish them through bankon.pythai.net's identity layer for cross-agent coordination.

The recommended retry strategy synthesizes the official guidance with field experience. Always parse the JSON body, never trust HTTP status alone (especially while streaming). Distinguish bucket types: on `limit_rpd/*` sleep until `X-RateLimit-Reset` (or until next 00:00 UTC, whichever is sooner); on `limit_rpm/*` sleep $\sim$ 60 s with jitter; on `Provider returned error` switch to a different provider via `provider.ignore` and retry immediately. Use exponential backoff with jitter (`min(60s, 2^attempt) + random(0, 1s)`), capped at 5 attempts) for transient

5xx. **Self-throttle proactively at ≤ 18 RPM** because failed attempts count against your daily cap. Use `(models: [primary, secondary:free, tertiary:free])` for OpenRouter's automatic model fallback — pricing flows from the model that ultimately succeeded. Respect Cloudflare's DDoS layer by spacing burst regenerations ≥ 250 ms apart even when within RPM. For production-critical paths, abandon `(:free)` entirely. `(OpenAI + 2)`

Provider routing, model routing, caching, tools, structured outputs, vision, reasoning, web search, PDFs, transforms

The `(provider)` object inside the request body controls upstream selection. Its full schema is `{order: string[], allow_fallbacks: bool=true, require_parameters: bool=false, data_collection: "allow"|"deny", zdr: bool, enforce_distillable_text: bool, only: string[], ignore: string[], quantizations: ("int4"|"int8"|"fp4"|"fp6"|"fp8"|"fp16"|"bf16"|"fp32"|"unknown")[], sort: "price"|"throughput"|"latency" | {by, partition: "model"|"none"}, max_price: {prompt, completion, request, image}, preferred_max_latency: {p50, p90, p99}, preferred_min_throughput: {p50, p90, p99}}`. Three default behaviors matter: setting `(order)` or `(sort)` **disables** OpenRouter's price-weighted load balancing; `(preferred_*)` fields *deprioritize* misses but don't exclude (only `(max_price)` is a hard filter); per-request `(zdr: true)` ORs with the account-level toggle and cannot be turned off per-request. Provider slugs match by base (`(google-vertex)` matches all regions) or by exact endpoint (`(google-vertex/us-east5)`, `(deepinfra/turbo)`). The shortcut suffixes on a model slug are equivalent to provider sort modes: `(:nitro) = (sort: "throughput")`, `(:floor) = (sort: "price")`, `(:exacto) =` quality-weighted ordering tuned for tool-calling reliability. Auto-Exacto, on by default since March 10 2026 for tool-calling requests on supported models (GLM-4.7, GLM-5, DeepSeek V3.2, gpt-oss-120b), reorders providers every ~5 minutes based on real-time tool-call telemetry — OpenRouter reports an 88 % tool-call error reduction on GLM-5 and 36 % on gpt-oss-120b, at the cost of occasionally favoring a pricier provider (use `(:floor)` to override). `(Translate + 14)`

Model-level routing has three flavors. The `(models: [...])` array attaches an ordered fallback list — any error (rate limit, moderation, downtime, content filter) walks to the next entry; pricing reflects the model that succeeded; `(response.model)` and `(response.provider)` tell you which. The legacy `(route: "fallback")` does the same. The Auto Router lives at `(model: "openrouter/auto")` and is powered by NotDiamond — a classifier picks an optimal model per prompt from a curated pool (Claude Sonnet/Opus 4.5-4.7, GPT-5.1, Gemini 3.1 Pro, DeepSeek 3.2, etc.), pays the chosen model's rate with no surcharge, and supports filtering via `(plugins: [{id: "auto-router", allowed_models: ["anthropic/*", "openai/gpt-5.1"]}])`. The Free Models Router at `(model: "openrouter/free")` randomly picks from currently-healthy free models. mindX should expose Auto Router as a "best-effort cognitive" route in the model registry but log the actual model used for billing and reproducibility audit. `(OpenRouter + 5)`

Prompt caching is the single largest cost lever. OpenRouter implements provider-sticky routing: subsequent requests with identical opening tokens hash to the same provider when cache reads are cheaper than fresh prompt costs, *unless* you've pinned `(provider.order)`, which disables stickiness. Per-response `(usage.prompt_tokens_details.cached_tokens)` (read), `(cache_write_tokens)`, and `(usage.cache_discount)` (which can be negative when a write premium is being charged) document hits. **OpenAI, Grok, Moonshot, DeepSeek, and Gemini 2.5 Flash/Pro cache automatically** — no `(cache_control)` annotations, free writes, reads at $0.10\times-0.50\times$ input price depending on model. **Anthropic Claude requires explicit `(cache_control: {type: "ephemeral"})` breakpoints** with 5-minute ($1.25\times$ write, $0.10\times$ read) or 1-hour ($2\times$ write, $0.10\times$ read) TTLs, minimum cacheable prompts of 1 024–4 096 tokens depending on model, and a hard ceiling of **4 explicit breakpoints**. The simplest Anthropic caching pattern is a top-level `(cache_control: {type: "ephemeral"})` field on the request body — but this **only routes to the Anthropic provider directly**, not Bedrock or Vertex; for those use per-content-block breakpoints. For Gemini, the `(systemInstruction)` is immutable when cached, so dynamic prefixes must move to a later user message rather than appending to the first system message — a common cache-busting bug. (OpenRouter + 8)

Tool calling uses the OpenAI shape with `(parallel_tool_calls: true)` by default; OpenRouter normalizes Anthropic's XML and Gemini's structured function calls behind the scenes. The tool definitions **must be passed on every request in the conversation loop**, not just the first one. Streaming tool calls arrive piecewise in `(delta.tool_calls[])` — the `(id)` and `(function.name)` come on the first chunk, then `(function.arguments)` streams token-by-token across many chunks; clients accumulate by `(index)`. Strict-mode validation requires the `(x-anthropic-beta: structured-outputs-2025-11-13)` header on Claude or OpenRouter silently strips `(strict: true)`. Many `(:free)` slugs return `(404 No endpoints found that support tool use)` (e.g., `(deepseek/deepseek-r1-0528:free)`) or reject `(tool_choice: "required")` (e.g., `(z-ai/glm-4.5-air:free)` — accepts only `(auto)`); for tool-using agents mindX should default to `(:exacto)` or paid endpoints. (OpenRouter + 4)

Structured outputs comes in two flavors: `(response_format: {type: "json_object"})` for valid-JSON-but-no-schema, and `(response_format: {type: "json_schema", json_schema: {name, strict, schema}})` for true schema enforcement. Combine with `(provider.require_parameters: true)` to force routing only to providers that actually enforce schemas. OpenAI GPT-4o and later, all Gemini, Anthropic Sonnet 4.5 / Opus 4.1+, and Fireworks-hosted open-source models enforce strictly; for others use the **(response-healing) plugin** (`(plugins: [{id: "response-healing"}])`) which fixes JSON syntax (not schema mismatches) at <1 ms CPU overhead and reduces defect rates 80–99 %. Note: GPT-5.2 and GLM-5 silently drop reasoning content when combined with `(json_schema)` — for reasoning-preserving structured output, use tool calling instead. (OpenRouter + 4)

Vision uses the OpenAI multimodal content-parts schema with text-before-image ordering for best parsing. **Reasoning tokens** unify across providers via `(reasoning: {effort?,`

`max_tokens?, exclude?, enabled?}`). Effort levels

`(xhigh/high/medium/low/minimal/none)` map to ratios of `max_tokens` ($\approx$

0.95/0.80/0.50/0.20/0.10) per provider; OpenAI o-series accepts effort but **does not return**

reasoning content (only summaries), Anthropic 3.7+ returns it via `reasoning_details`, Grok

and DeepSeek R1 return it in the `reasoning` field, Gemini 2.5/3.x maps to

`thinkingBudget/thinkingLevel`. `max_tokens` **must strictly exceed the reasoning budget** or

the final answer truncates. To preserve reasoning continuity across a tool-calling loop on Claude

(required for `interleaved-thinking-2025-05-14`), the entire `reasoning_details[]` array

must be passed back unmodified on the assistant message in the same order as received.

Reasoning tokens are billed as output tokens, even when `exclude: true`. `(openrouter + 7)`

Web search has two forms: the legacy plugin `plugins: [{id: "web", engine:`

`"exa"|"firecrawl"|"parallel"|"native", max_results: 5, include_domains,`

`exclude_domains}]` plus the `:online` model-slug shortcut (equivalent to `openrouter/auto` +

`web` plugin), and the newer recommended server tool `tools: [{type:`

`"openrouter:web_search"}]` which lets the model decide when to search and works with both

Chat Completions and Responses APIs. Pricing: Exa/Parallel charge **\$4 per 1 000 results** (\$0.02

per request at default `max_results: 5`), Firecrawl uses your BYOK Firecrawl credits, native is

provider passthrough. **Web search charges even on `:free` models.** Citations standardize into

`message.annotations[]` with `{type: "url_citation", url_citation: {url, title,`

`content, start_index, end_index}}`. **PDFs** work on every model via `{type: "file", file:`

`{filename, file_data}}` content parts, with engines selected by `plugins: [{id: "file-`

`parser", pdf: {engine: "native"|"mistral-ocr"|"cloudflare-ai"}}]`. Skip-reparse:

response carries an `annotations: [{type: "file", file: {hash, name, content: [...]}]}`

block — pass it back on subsequent requests in the same conversation to use the cached parse

instead of re-OCR-ing. `(openrouter + 9)`

Message transforms: `transforms: ["middle-out"]` is the only available transform and is **on**

by default for endpoints with ≤ 8 K context, off for larger windows — set `transforms: []` or

`plugins: [{id: "context-compression", enabled: false}]` to disable. It removes/truncates

middle messages until the prompt fits, keeping half from start and half from end. Inspired by the

Lost-in-the-Middle paper. **Other production features** worth knowing: **Zero Completion**

Insurance is always on, charges nothing when the response is empty or `finish_reason:`

`"error"`. **Response caching** (beta, request-level, distinct from prompt caching) is enabled per-

request via `X-OpenRouter-Cache: true`, `X-OpenRouter-Cache-TTL: 300` (1-86 400 s), and `X-`

`OpenRouter-Cache-Clear: true` to force-refresh; cache hits cost zero credits and replay through

the streaming pipeline with a fresh gen-id. **Presets** at `model: "@preset/<slug>"` or

`"anthropic/claude-sonnet-4.5@preset/code-reviewer"` encapsulate routing, system

prompts, and parameters server-side. **Service tiers** pass through to providers exposing latency

tiers (e.g., OpenAI flex vs priority). `(openrouter + 7)`

Python integration: the OpenAI SDK pattern as the production default

There is an official OpenRouter Python SDK in beta (`pip install openrouter`, `from openrouter import OpenRouter`) but it is auto-generated from OpenAPI and not yet battle-tested. The production-grade pattern, and what OpenRouter's own quickstart shows, is the **OpenAI Python SDK with a custom `base_url`**. The complete sync invocation is: `OpenRouter`

```
python
```

```

import os
from openai import OpenAI

client = OpenAI(
    base_url="https://openrouter.ai/api/v1",
    api_key=os.environ["OPENROUTER_API_KEY"],
    default_headers={
        "HTTP-Referer": "https://mindx.pythai.net",
        "X-Title": "mindX",
    },
    timeout=60.0,
    max_retries=0, # we manage retries ourselves
)

completion = client.chat.completions.create(
    model="anthropic/claude-sonnet-4.5",
    messages=[
        {"role": "system", "content": "You are a mindX cognitive agent."},
        {"role": "user", "content": "Decompose the following task into subgoals..."}
    ],
    temperature=0.2,
    max_completion_tokens=2048,
    extra_body={
        "provider": {
            "order": ["anthropic", "google-vertex"],
            "allow_fallbacks": True,
            "require_parameters": True,
            "data_collection": "deny",
            "zdr": True,
        },
        "models": ["anthropic/claude-sonnet-4.5", "openai/gpt-5.2", "meta-llama/llama-3.1-70b-instruct"],
        "transforms": ["middle-out"],
        "reasoning": {"effort": "medium"},
    },
    extra_headers={"x-session-id": "mindx-session-abc123"},
)

print(completion.choices[0].message.content)
print("gen-id:", completion.id, "actual model:", completion.model)

```

The critical detail is that **OpenRouter-only fields** (`provider`, `models`, `transforms`, `reasoning`, `plugins`, `usage`, `route`, `zdr`) must go in `extra_body`, because the OpenAI SDK

strips unknown top-level parameters. Per-request header overrides go in `extra_headers`. The async equivalent uses `from openai import AsyncOpenAI` with `await client.chat.completions.create(...)` and `await client.close()`. Streaming consumption with the SDK looks like: `Snyk`

```
python

stream = await client.chat.completions.create(
    model="openai/gpt-5.2",
    messages=[{"role": "user", "content": "..."}],
    stream=True,
)
generation_id = None
async for chunk in stream:
    if generation_id is None and chunk.id:
        generation_id = chunk.id
    if chunk.choices and chunk.choices[0].delta.content:
        print(chunk.choices[0].delta.content, end="" flush=True)
    if getattr(chunk, "usage", None):
        # final usage chunk – log cost
        log_usage(generation_id, chunk.usage)
```

For tool-call streaming, accumulate by `delta.tool_calls[i].index`: the first chunk carries `id` and `function.name`, subsequent chunks stream `function.arguments` token-by-token. For raw httpx streaming when avoiding the SDK (e.g. when mindX needs absolute control over SSE parsing in a coroutine), the canonical loop is `async with client.stream("POST", url, headers=..., json=...) as resp: async for raw_line in resp.aiter_lines()`, where each non-empty non-comment `data:` line is JSON-parsed, `[DONE]` terminates, and any chunk with a top-level `error` field is raised — this is the pattern the **streaming consumption coroutine** addendum (`openrouter_stream_coroutine.md`) implements in full.

Generation cost is captured two ways. The inline path: every response now includes `usage.cost` in the JSON body (OpenRouter populates it automatically since the deprecation of `usage: {include: true}`); zero extra HTTP calls required. The audit path: `GET /api/v1/generation?id=<gen_id>` returns the canonical authoritative record with **37 fields** including `total_cost`, `cache_discount`, `upstream_inference_cost` (BYOK only), `latency`, `moderation_latency`, `generation_time`, `tokens_prompt`/`tokens_completion` (GPT-tokenizer normalized) and `native_tokens_prompt`/`native_tokens_completion` (provider's own tokenizer — **what billing actually uses**), `provider_responses[]` enumerating every retry attempt, `is_byok`, `cancelled`, `api_type`, `router`, `request_id`, `app_id`, `external_user`. There is a **1-2 second indexing delay** before the row becomes queryable, so mindX's cost-tracking coroutine

(`openrouter_cost_coroutine.md`) fires asynchronously with exponential backoff (0.5 s → 1 s → 2 s → 4 s, 6 attempts). ([GitHub + 5](#))

Error handling maps OpenRouter HTTP codes to the OpenAI SDK's exception hierarchy:

400→(`BadRequestError`), 401→(`AuthenticationError`), 402→(`APIStatusError(status=402)`), 403→(`PermissionDeniedError`), 404→(`NotFoundError`), 408→(`APITimeoutError`), 429→(`RateLimitError`), 500/502/503→(`InternalServerError`)/(`APIError`). mid-stream errors require manual SSE parsing because HTTP stays 200. The recommended retry decorator wraps these with tenacity's (`retry_if_exception_type((RateLimitError, APITimeoutError, APIConnectionError, InternalServerError, APIError))`), (`wait_random_exponential(min=1, max=60)`), (`stop_after_attempt(6)`). **Production wrappers** add an (`aiolimiter.AsyncLimiter(rps=8, time_period=1.0)`) for client-side throttling under the 20 RPM cap, an (`asyncio.Semaphore(max_concurrency=20)`) to bound in-flight requests, and a (`pybreaker.CircuitBreaker(fail_max=5, reset_timeout=30)`) per (`(model, provider)`) pair to fail fast during upstream outages. ([Reintech](#))

LangChain integration has no first-party (`langchain-openrouter`) package; the de-facto pattern is (`ChatOpenAI(base_url="https://openrouter.ai/api/v1", api_key=..., default_headers={"HTTP-Referer": ..., "X-Title": ...})`) with OpenRouter extras going into (`extra_body=`). A thin (`ChatOpenRouter(ChatOpenAI)`) subclass that reads (`OPENROUTER_API_KEY`) is the community standard. **LlamaIndex** has (`llama_index.llms.openrouter.OpenRouter`) (PyPI: `llama-index-llms-openrouter`); not all OR fields are exposed as native kwargs (run-llama/llama_index #17751 tracks this), so for full control use the OpenAI SDK pattern.

TypeScript/Node: either the (`openai`) SDK with (`baseUrl: "https://openrouter.ai/api/v1"`) or the official (`@openrouter/ai-sdk-provider`) for Vercel AI SDK (`(createOpenRouter({apiKey}))`) which exposes OR extras three ways (`(providerOptions.openrouter, extraBody)` on the model factory, (`extraBody`) in model settings). **Rust:** community crates (`openrouter_api`) (socrates8300) and (`openrouter-rs`), or (`async-openai`)/(`openai-api-rs`) with a custom base URL. **Go:** the official OpenRouter Go SDK at (`openrouter.ai/docs/sdks/go-sdk/`), or (`sashabaranov/go-openai`) with (`cfg.BaseURL = "https://openrouter.ai/api/v1"`). **Pydantic-AI** has a native (`pydantic_ai.models.openrouter`) provider. **LiteLLM** prefixes routes with (`openrouter/...`) and exposes uniform exception types — useful as a proxy in front of OpenRouter when mindX needs uniform error handling across both OpenRouter and direct providers. ([Medium + 8](#))

mindX adapter architecture

mindX should treat OpenRouter as one **registered LLM backend** in a pluggable backend registry, not as the universal default. The architectural rationale is sovereignty: PYTHAI's cypherpunk2048 standard mandates no proprietary lock-in, and a hard dependency on OpenRouter would violate that. The right shape is a (`mindx.llm.backends.openrouter`) module implementing a (`LLMBackend`) protocol that the cognitive layer can swap with

`mindx.llm.backends.anthropic_direct`, `mindx.llm.backends.local_vllm`, or `mindx.llm.backends.bankon_byok` without code changes downstream. The protocol (a `typing.Protocol` with `complete()`, `stream()`, `embed()`, `tool_loop()`, `cost_of()` async methods) lives at `mindx/llm/protocol.py`; the OpenRouter implementation at `mindx/llm/backends/openrouter/__init__.py` flat snake_case per cypherpunk2048.

The OpenRouter backend module decomposes into eight files. `client.py` holds an `OpenRouterClient` class wrapping `AsyncOpenAI` with `default_headers={"HTTP-Referer": "https://mindx.pythai.net", "X-Title": "mindX", "X-OpenRouter-Categories": "agentic-cognition"}`, an `aiolimiter.AsyncLimiter` instance per account, an `asyncio.Semaphore` for concurrency, and a `pybreaker.CircuitBreaker` per (model, provider) pair. `registry.py` holds `ModelRegistry` which polls `GET /api/v1/models` every six hours, caches results in mindX's storage with a `last_seen_at` timestamp per slug, classifies each model by capability (`tools`, `vision`, `reasoning`, `structured_outputs`, `caching`, `web_search`), tier (`free`, `paid`, `byok`), and quality band (`flagship`, `standard`, `cheap`, `experimental`), and emits deprecation events to bankon.pythai.net's identity layer when a slug disappears. `router.py` maps an agent's task description to a model selection: `cognitive_critical` → `openai/gpt-5.2` with `provider.sort: "throughput"`, `cognitive_default` → `openrouter/auto` with `allowed_models: ["anthropic/*", "openai/gpt-5.1", "google/gemini-3.1-pro"]`, `agent_tool_loop` → `:exacto-flagged paid slug`, `agent_self_critique` → `meta-llama/llama-3.3-70b-instruct:free` with `provider.ignore: ["Targon"]`, `batch_classification` → `openrouter/free`, `vision` → `google/gemma-4-31b-it:free or paid Claude/Gemini`, `coding` → `qwen/qwen3-coder:free or paid GPT-5.2 Codex`, `embedding` → `bypass OpenRouter, route to direct provider`. The router's selection is logged with `model`, `provider`, `tier`, `selected_at`, `reason` for reproducibility.

`fallback.py` assembles the `models[]` array per task class. The free-tier fallback chain is `[primary:free, secondary:free, openrouter/free]`; the paid chain is `[primary, anthropic/claude-sonnet-4.5, openai/gpt-5.2, meta-llama/llama-3.3-70b-instruct:free]` (with the free tail as a last-resort degradation); the BYOK chain is `[byok-primary, openrouter-shared-primary, fallbacks]`. `cache_policy.py` decides `cache_control` annotations per request: Anthropic gets `{type: "ephemeral", ttl: "1h"}` on system prompts ≥ 4 096 tokens, OpenAI/Gemini/Grok/Moonshot rely on automatic caching, GLM and Z.AI get explicit per-block breakpoints. `cost_tracker.py` runs the cost-tracking coroutine (see addendum), emitting `(gen_id, model, provider, native_tokens_prompt, native_tokens_completion, total_cost, cache_discount, latency, is_byok)` tuples to mindX's append-only ledger which bankon.pythai.net then reconciles against tenant billing. `rate_limit_tracker.py` runs the rate-limit coroutine (see addendum), maintaining client-side counters per `(account, model_class)` and proactively shedding load before hitting OpenRouter limits. `stream_consumer.py` runs the streaming consumption coroutine (see

addendum), tolerating SSE comment heartbeats and surfacing mid-stream errors as exceptions. `extension.py` holds the Module Extension Coroutine plumbing (see addendum) for adding new providers (e.g., a future Azure-direct provider, a future on-prem Llama deployment) without modifying the core backend.

Configuration lives at `mindx/config/openrouter.toml` with keys `api_key_env_var = "OPENROUTER_API_KEY"`, `management_key_env_var = "OPENROUTER_MANAGEMENT_KEY"`, `referer = "https://mindx.pythai.net"`, `title = "mindX"`, `default_categories = ["agentic-cognition"]`, `rps_limit = 8`, `max_concurrency = 20`, `circuit_breaker_threshold = 5`, `circuit_breaker_reset_seconds = 30`, `model_registry_poll_hours = 6`, `cost_lookup_max_attempts = 6`, `default_provider_data_collection = "deny"`, `default_provider_zdr = true`, `default_quantizations = ["fp16", "bf16", "fp8"]`, `default_max_price_prompt = 5.0`, `default_max_price_completion = 15.0`. **Secrets** are sealed Podman secrets (`mindx-openrouter-api-key`, `mindx-openrouter-management-key`), never environment variables in production, never committed to git. **Dependencies** are pinned in `pyproject.toml` for Python ≥ 3.12 : `openai>=1.50`, `httpx>=0.27`, `tenacity>=9.0`, `aiolimiter>=1.2`, `pybreaker>=1.2`, `pydantic>=2.9`, `tomli>=2.0` for config, plus `langfuse>=2.50` if observability is enabled.

BYOK integration with bankon.pythai.net: a tenant's identity page exposes a "Bring Your Own LLM Provider" section that writes the upstream provider key into OpenRouter via the management API. mindX's adapter detects the tenant's `byok_enabled` flag in their identity payload and switches the request's behavior — when enabled, requests use the tenant's dedicated OpenRouter inference key (which OpenRouter's BYOK system routes to the tenant's upstream credentials first), with `is_byok: true` flowing back in the response for billing reconciliation. **AgenticPlace integration:** agents listed in the marketplace declare a `llm_requirements` manifest (`{tier: "free"|"paid"|"byok", capabilities: ["tools", "vision", ...], min_context: 65536, fallback_acceptable: true}`); mindX's router consumes the manifest to pick the right OpenRouter route at agent invocation time.

Observability: configure Langfuse via OpenRouter's no-code "OpenRouter Broadcast" integration (Langfuse keys entered in OpenRouter settings → automatic tracing of every request) or use Langfuse's OpenAI wrapper for nested tracing. Helicone is in maintenance mode post-Mintlify acquisition and should be avoided for new deployments. `Langfuse + 2`

Production reliability: the gotchas that bite

Several behaviors only emerge under load and mindX must defend against each. **Free-model 404 on tool use** (`deepseek/deepseek-r1-0528:free` → "No endpoints found that support tool use"): the tool-loop router must never select `:free` slugs unless `supported_parameters` includes `tools`. **Free-tier 429 storms during peak hours** (Chutes saturating from 14:00–22:00 UTC): the rate-limit coroutine should track 429-rate per `(model, provider)` and trigger automatic provider blacklisting via `provider.ignore` when the rate exceeds 5 % over a 5-

minute window. **Mid-stream errors via SSE:** the stream consumer must inspect every chunk for a top-level `error` field even when HTTP is 200 — naive `httpx`/`requests` consumers that only check status codes will silently drop responses, and litellm has a long-standing issue (BerriAI/litellm #9035) with this exact failure mode. **Empty completions billed as zero:** Zero Completion Insurance handles billing, but mindX must still treat zero-token completions as transient errors and retry on a different provider — they often indicate a cold-start or scaling issue on the upstream. **Context window mismatch between providers:** `qwen/qwen-2.5-coder-32b-instruct` exposes 33 K on DeepInfra and 128 K on Hyperbolic; `deepseek/deepseek-chat-v3.1:free` lists 163.8 K but routes to DeepInfra at 64 K (Roo-Code #7952). The model registry must store per-provider context windows and the router must use the *minimum* across the fallback chain when chunking inputs. **The `:free` suffix gotcha:** hardcoding `deepseek/deepseek-r1` instead of `deepseek/deepseek-r1:free` triggers paid billing; some agents (opencode #1050, goose #3054) strip the suffix when constructing fallback chains, breaking free-tier intent. mindX's router must lint slug strings against the registry's tier flag.

GitHub + 4

The `$10 unlock` is mandatory for any production deployment — it's the cheapest reliability lever in OpenRouter and lifts the free RPD from 50 to 1 000 forever. **Strict-mode tool calling** requires the `x-anthropic-beta: structured-outputs-2025-11-13` header on Claude or `strict: true` is silently dropped. **Provider sticky routing for caching** is disabled when `provider.order` is set — accept lower cache hit rates or use `provider.sort.partition: "none"`. **Multi-account does not raise free-tier limits** — capacity is governed globally per account org, so spinning up sibling accounts to bypass the 1 000 RPD cap is wasted effort; deposit credits or use BYOK instead. **Token counts use the upstream provider's tokenizer** (the response `usage` is authoritative for billing); don't assume cl100k-equivalence. **`provider.allow_fallbacks: false` makes free-tier reliability *strictly worse*** — only set it when you must pin a specific provider for compliance reasons. **Anthropic `cache_control` at top level only routes to anthropic.com** (Bedrock and Vertex are silently excluded); for those use per-content-block breakpoints. OpenRouter + 3

The 2025–2026 changelog mindX must internalize: July 2025 free-tier reshuffle removed many providers; October 2025 introduced `:exacto`; October 2025 made the first 1 M BYOK requests/month free; December 2025 added the Response Healing plugin and `openrouter/free` meta-router; November 13 2025 broke strict-mode tool calls without the new header; March 2026 turned on Auto-Exacto by default for tool-calling on supported models; February 2026 confirmed 39 free-model deprecations. Outages: a major Cloudflare+GCP cascade on June 12 2025; running outage tracking via `status.openrouter.ai` (no historical data) plus aggregators like statusgator.com/services/openrouter (46+ outages tracked since February 2025) and incidenthub.cloud/status/openrouter. OpenRouter + 5

Conclusion: OpenRouter as a sovereign-friendly LLM commodity layer

OpenRouter solves the heterogeneity problem cleanly — one OpenAI-shaped API, normalized errors, transparent failover, automatic prompt caching, free-tier headroom, BYOK without surrendering keys — but it solves no reliability problem on its own. mindX must layer **client-side rate limiting** (≤ 18 RPM, semaphore-bounded concurrency), **per-provider circuit breakers** (5-fail/30-s reset), **dynamic model registry polling** (six-hour cadence with deprecation alerts), **strict tier discipline** (`:free` never on user-facing critical paths), and **complete provenance logging** (`gen_id`, `model`, `provider`, `tier`, `cost`, `latency` on every request) to extract reliable behavior from a fundamentally probabilistic backplane. The single highest-leverage operational decision is the \$10 credit deposit that lifts free RPD by 20×; the second is the model-registry polling that makes the system self-healing against OpenRouter's aggressive churn; the third is parsing the JSON-body `error.metadata.headers` rather than HTTP headers for rate-limit semantics. Done correctly, OpenRouter becomes mindX's commodity LLM layer — swappable, auditable, and entirely consistent with cypherpunk2048's no-lock-in principle, because the `LLMBackend` protocol means the day OpenRouter stops being the right answer is the day mindX swaps it out without touching agent code.

Lowercase addendum file inventory

The following addendum files complete this document and live alongside `OPENROUTER.md` in mindX's docs directory. Each implements a single coroutine pattern referenced above.

`openrouter_async_client_coroutine.md` — implements `OpenRouterClient` as an async context manager wrapping `AsyncOpenAI` with `aiolimiter.AsyncLimiter(rps=8, time_period=1.0)`, `asyncio.Semaphore(20)`, tenacity-decorated `_create()` retrying on `(RateLimitError, APITimeoutError, APIConnectionError, InternalServerError, APIError)` with `wait_random_exponential(min=1, max=60)` and `stop_after_attempt(6)`. Exposes `chat(model, messages, **kwargs)`, `stream(model, messages, **kwargs)`, `embed(model, texts)`, and `aclose()`. Reads config from `mindx/config/openrouter.toml`. Resolves API key from sealed Podman secret at boot.

`openrouter_module_extension_coroutine.md` — defines the `LLMBackend` Protocol at `mindx/llm/protocol.py` with async methods `complete(model, messages, **kwargs) -> Completion`, `stream(...) -> AsyncIterator[Chunk]`, `embed(...)`, `tool_loop(...)`, `cost_of(gen_id) -> Cost`. Implements registration via `mindx.llm.registry.register("openrouter", OpenRouterBackend)` so additional providers (`anthropic_direct`, `local_vllm`, `bankon_byok`) plug in without core modifications. The extension coroutine (`async def extend_with_provider(slug, factory)`) adds new OpenRouter-routed providers at runtime by validating against `GET`

`/api/v1/models/{author}/{slug}/endpoints`), registering capability flags, and injecting fallback chains.

`openrouter_rate_limit_coroutine.md` — runs as a background task per OpenRouter account. Maintains in-memory counters keyed by `(account, model_tier, model_slug)` with sliding-window RPM tracking and rolling daily counter reset at 00:00 UTC. Polls `GET /api/v1/key` every 60 seconds to refresh `is_free_tier`, `usage_daily`, `limit_remaining`. Subscribes to a `pre_request` hook on `OpenRouterClient` and **denies dispatch with `RateLimitWouldExceedError`** when projected RPM exceeds 18 or projected RPD exceeds (50 or 1000 - 5). On 429 receipt, parses `error.metadata.headers["X-RateLimit-Reset"]` (epoch ms!) and schedules account-wide backoff. Tracks per-`(model, provider)` 429-rate and triggers `provider.ignore` injection when rate > 5 % over 5 minutes.

`openrouter_cost_coroutine.md` — fires after every completion as a fire-and-forget task. Captures `gen_id` from the response. Polls `GET /api/v1/generation?id=<gen_id>` with exponential backoff (0.5s → 1s → 2s → 4s → 8s → 16s, max 6 attempts) until `data.total_cost` is not None. Emits `CostRecord(gen_id, model, provider, native_tokens_prompt, native_tokens_completion, total_cost, cache_discount, generation_time_ms, latency_ms, moderation_latency_ms, is_byok, upstream_inference_cost, request_id, app_id, external_user, created_at)` to mindX's append-only cost ledger. Bankon.pythai.net consumes the ledger nightly to reconcile tenant billing and flag anomalies (cost > 2σ above tenant baseline). Handles 404/425 (not yet indexed) as transient; 401/403 as fatal.

`openrouter_stream_coroutine.md` — implements raw httpx streaming for absolute SSE control. The coroutine `async def stream_chat(model, messages, **kwargs) -> AsyncIterator[StreamEvent]` opens `client.stream("POST", "https://openrouter.ai/api/v1/chat/completions", ...)`, captures `X-Generation-Id` header, then iterates `resp.aiter_lines()` skipping empty lines and lines starting with `:` (SSE comments — handles `: OPENROUTER PROCESSING`), parses `data: ...` lines as JSON, terminates on `[DONE]`, raises `OpenRouterStreamError` on any chunk with a top-level `error` field, accumulates tool-call deltas by `index`, yields a final `UsageEvent(usage)` from the last chunk before `[DONE]`. Implements 30-second no-token watchdog that aborts the stream and triggers fallback. Yields typed events: `ContentDelta`, `ToolCallDelta`, `ReasoningDelta`, `UsageEvent`, `FinishEvent`. Caller composes these via `async for event in stream_chat(...)` for natural backpressure.